Question 033 | Question bank page 4 | Q.1

MVC Application for Basic Arithmetic Operations

Project type: ASP.NET MVC / ASP.NET Core

Steps to perform

1. Create the appropriate MVC project in Visual Studio (match the source framework).

2. Add the model, controller and Razor view files in their named folders.

3. Configure routing, dependencies and database/authentication services if needed.

4. Build and launch with Ctrl+F5.

5. Enter test data in the browser and verify the expected response.

Closest supplied practical: Create an MVC application to perform basic Arithmetic Operations.

Source: Dot_Net_Practicals_1_to_5_.pdf

Provided creation instructions and source files

```
Creation steps: Create the MVC Application named ArthimeticMVC. Add the listed files at the
locations below and enter
the corresponding code.
Filename: Index.cshtml    Location: Views/ (MVC view folder)
@model ArithmeticMVC.Models.ArithmeticModel
@{
ViewBag.Title = "Arithmetic Operations";
}
<h2>Arithmetic Operation</h2>
@using (Html.BeginForm())
{
<table>
    <tr>
        <td>Frist Number</td>
        <td>@Html.TextBoxFor(m=>m.Number1)</td>
    </tr>
    <tr>
        <td>Second Number</td>
        <td>@Html.TextBoxFor(m=>m.Number2)</td>
    </tr>
    <tr>
        <td>Operation</td>
        <td>
            @Html.DropDownListFor(m=>m.Operation,
            new SelectList(new[]
            {
            "Add",
            "Subtract",
            "Multiply",
            "Divide"
            }))
        </td>
    </tr>
    <tr>
        <td></td>
        <td>
            <input type="submit" value="Calculate"/>
        </td>
    </tr>
</table>
}
@if(!string.IsNullOrEmpty(Model.ErrorMessage))
{
<h3 style="color:red">@Model.ErrorMessage</h3>
}
else if(Model.Operation!=null)
{
<h3>Result=@Model.Result</h3>
}
Filename: HomeController.cs    Location: Controllers/HomeController.cs
public class HomeController : Controller
{
    public ActionResult Index(ArithmeticModel model)
    {
        if (ModelState.IsValid)
        {
```

```
            switch (model.Operation)
            {
            case "Add":
                model.Result = model.Number1 + model.Number2;
                break;
            case "subtract":
                model.Result = model.Number1 - model.Number2;
                break;
            case "Multiply":
                model.Result = model.Number1 * model.Number2;
                break;
            case "Divide":
                if (model.Number2 == 0)
                {
                    model.ErrorMessage = "Divison by zero is not allowed.";
                }
                else
                {
                    model.Result = model.Number1 / model.Number2;
                }
                break;
            }
        }
        return View(model);
    }
}
Filename: ArithmeticModel.cs    Location: Models/ArithmeticModel.cs
using System.ComponentModel.DataAnnotations;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Web;
namespace ArithmeticMVC.Models
{
    public class ArithmeticModel
    {
        public double N
umber1 { get; set; }

        public double Number2 { get; set; }
        public string Operation { get; set; }
        public double Result { get; set; }
        public string ErrorMessage { get; set; }
    }
}
```

Expected result / verification

The application should meet the exact question requirements. Record actual output or screenshots after running; no execution has been claimed in this manual.